Advanced Rust (2026): Unsafe Rust

Lecture 5 · Alternative

Lukáš Hozda

Spring 2026

MFF CUNI

Unsafe Rust

Unsafe Rust

- Rust has safe Rust and unsafe Rust

Unsafe Rust

- Rust has safe Rust and unsafe Rust
- `unsafe` does not disable the type system

Unsafe Rust

- Rust has safe Rust and unsafe Rust
- `unsafe` does not disable the type system
- It does let you perform operations the compiler cannot justify on its own

Unsafe Rust

- Rust has safe Rust and unsafe Rust
- `unsafe` does not disable the type system
- It does let you perform operations the compiler cannot justify on its own
- The price is that proof obligations move from compiler to programmer

Safe Rust

- Safe code cannot dereference dangling references

Safe Rust

- Safe code cannot dereference dangling references
- Safe code cannot observe uninitialized values through safe APIs

Safe Rust

- Safe code cannot dereference dangling references
- Safe code cannot observe uninitialized values through safe APIs
- Safe code cannot violate reference aliasing rules

Safe Rust

- Safe code cannot dereference dangling references
- Safe code cannot observe uninitialized values through safe APIs
- Safe code cannot violate reference aliasing rules
- Safe code cannot cause unsynchronized write races

Safe Rust

- Safe code cannot dereference dangling references
- Safe code cannot observe uninitialized values through safe APIs
- Safe code cannot violate reference aliasing rules
- Safe code cannot cause unsynchronized write races
- Safe code can still leak memory, deadlock, or compute nonsense

Forms of `unsafe`

- `unsafe fn`: caller must satisfy extra preconditions

Forms of `unsafe`

- `unsafe fn`: caller must satisfy extra preconditions
- `unsafe trait`: implementer promises extra semantic guarantees

Forms of `unsafe`

- `unsafe fn`: caller must satisfy extra preconditions
- `unsafe trait`: implementer promises extra semantic guarantees
- `unsafe impl`: you assert that those guarantees really hold

Forms of `unsafe`

- `unsafe fn`: caller must satisfy extra preconditions
- `unsafe trait`: implementer promises extra semantic guarantees
- `unsafe impl`: you assert that those guarantees really hold
- `unsafe { ... }`: local escape hatch for specific operations

What `unsafe` Permits

- Dereferencing raw pointers

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions
- Accessing union fields

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions
- Accessing union fields
- Accessing or mutating mutable statics

What `unsafe` Permits

- Dereferencing raw pointers
- Calling unsafe functions
- Accessing union fields
- Accessing or mutating mutable statics
- Implementing unsafe traits

Why We Have Unsafe

- Some correct programs are beyond what the compiler can prove

Why We Have Unsafe

- Some correct programs are beyond what the compiler can prove
- Hardware and FFI live outside the compiler's model

Why We Have Unsafe

- Some correct programs are beyond what the compiler can prove
- Hardware and FFI live outside the compiler's model
- Concurrency primitives and low-level collections need raw capabilities

Why We Have Unsafe

- Some correct programs are beyond what the compiler can prove
- Hardware and FFI live outside the compiler's model
- Concurrency primitives and low-level collections need raw capabilities
- Performance sometimes matters, though this is the weakest reason unless measured

Why Unsafe Exists

```rust
const DEVICE_REGISTER_ADDRESS: *mut u32 = 0x1000_0000 as *mut u32;

unsafe fn write_to_device_register(value: u32) {
    std::ptr::write_volatile(DEVICE_REGISTER_ADDRESS, value);
}

fn main() {
    unsafe {
        write_to_device_register(0xABCD_EF01);
    }
}
```

Hardware Notes

- The compiler cannot know whether that address is mapped or valid

Hardware Notes

- The compiler cannot know whether that address is mapped or valid
- It cannot know whether concurrent access is allowed

Hardware Notes

- The compiler cannot know whether that address is mapped or valid
- It cannot know whether concurrent access is allowed
- On a hosted OS this is usually nonsense

Hardware Notes

- The compiler cannot know whether that address is mapped or valid
- It cannot know whether concurrent access is allowed
- On a hosted OS this is usually nonsense
- On bare metal this may be exactly the intended code

Another Hardware Example

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Safety depends on external facts about the machine

Another Hardware Example

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Safety depends on external facts about the machine
- HALs hide this from most users

Another Hardware Example

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Safety depends on external facts about the machine
- HALs hide this from most users
- The HAL itself still has to be written

Foreign Functions

```
extern "C" {
    fn strcpy(
        dest: *mut libc::c_char,
        src: *const libc::c_char,
    ) -> *mut libc::c_char;
}
```

- Rust cannot inspect what the foreign function actually does

Safe Wrapper Around FFI

```
fn safe_strcpy(dest: &mut [u8], src: &str) -> Result<(), &'static str> {
    let src_len = src.len();
    if dest.len() <= src_len {
        return Err("destination buffer is too small");
    }
```

```
    let c_src = std::ffi::CString::new(src).expect("CString::new failed");
    let src_ptr = c_src.as_ptr();
    let dest_ptr = dest.as_mut_ptr() as *mut libc::c_char;

    unsafe { strcpy(dest_ptr, src_ptr); }
    Ok(())
}
```

Concurrency Primitives

- `Mutex<T>` can manufacture `&mut T` from `&self`

Concurrency Primitives

- `Mutex<T>` can manufacture `&mut T` from `&self`
- The compiler cannot see the lock protocol in the machine state

Concurrency Primitives

- `Mutex<T>` can manufacture `&mut T` from `&self`
- The compiler cannot see the lock protocol in the machine state
- Unsafe code bridges from raw synchronization to safe API

Concurrency Primitives

- `Mutex<T>` can manufacture `&mut T` from `&self`
- The compiler cannot see the lock protocol in the machine state
- Unsafe code bridges from raw synchronization to safe API
- This is also why `UnsafeCell<T>` exists

Borrow Checker Limitations

```
fn next(buffer: &mut String) -> &str {
    loop {
        let event = parse(buffer);
        if true {
            return event;
        }
    }
}

fn parse(buffer: &mut String) -> &str { unimplemented!() }
```

- There are correct programs the borrow checker does not yet accept

Borrow Checker Limitations

```rust
fn next(buffer: &mut String) -> &str {
    loop {
        let event = parse(buffer);
        if true {
            return event;
        }
    }
}

fn parse(buffer: &mut String) -> &str { unimplemented!() }
```

- There are correct programs the borrow checker does not yet accept
- Reaching for `unsafe` here should be the exception

Borrow Checker Limitations

```rust
fn next(buffer: &mut String) -> &str {
    loop {
        let event = parse(buffer);
        if true {
            return event;
        }
    }
}

fn parse(buffer: &mut String) -> &str { unimplemented!() }
```

- There are correct programs the borrow checker does not yet accept
- Reaching for `unsafe` here should be the exception
- Usually a redesign is better than overriding the borrow checker

Borrow Checker Notes

- This is one place where people are tempted to say “I know better”

Borrow Checker Notes

- This is one place where people are tempted to say “I know better”
- Often they do not

Borrow Checker Notes

- This is one place where people are tempted to say “I know better”
- Often they do not
- Sometimes they do, but the burden of proof becomes entirely manual

Borrow Checker Notes

- This is one place where people are tempted to say “I know better”
- Often they do not
- Sometimes they do, but the burden of proof becomes entirely manual
- Future work such as Polonius may accept more patterns without unsafe

Performance Example

```rust
fn sum_vector_safe(v: &[i32]) -> i32 {
    let mut sum = 0;
    for &item in v.iter() {
        sum += item;
    }
    sum
}
```

```rust
fn sum_vector_unsafe(v: &[i32]) -> i32 {
    let mut sum = 0;
    let len = v.len();
    unsafe {
        let ptr = v.as_ptr();
        for i in 0..len {
            sum += *ptr.add(i);
        }
    }
    sum
}
```

Performance Notes

- Unsafe for performance is the most controversial category

Performance Notes

- Unsafe for performance is the most controversial category
- The benefit is often marginal

Performance Notes

- Unsafe for performance is the most controversial category
- The benefit is often marginal
- The maintenance cost is not marginal

Performance Notes

- Unsafe for performance is the most controversial category
- The benefit is often marginal
- The maintenance cost is not marginal
- Measure first, then decide

Values and UB

Compiler Assumptions

- The compiler assumes all code, including unsafe code, respects Rust's rules

Compiler Assumptions

- The compiler assumes all code, including unsafe code, respects Rust's rules
- It assumes no dangling dereference, no broken aliasing, no invalid values, and no unsynchronized write races

Compiler Assumptions

- The compiler assumes all code, including unsafe code, respects Rust's rules
- It assumes no dangling dereference, no broken aliasing, no invalid values, and no unsynchronized write races
- Optimizations are built on these assumptions

Compiler Assumptions

- The compiler assumes all code, including unsafe code, respects Rust's rules
- It assumes no dangling dereference, no broken aliasing, no invalid values, and no unsynchronized write races
- Optimizations are built on these assumptions
- If the assumptions are false, the optimizer is no longer constrained by source-level intuition

Valid Values

- A type is not just a bag of bits

Valid Values

- A type is not just a bag of bits
- `bool` only admits `0` and `1`

Valid Values

- A type is not just a bag of bits
- `bool` only admits `0` and `1`
- `!` admits no values at all

Valid Values

- A type is not just a bag of bits
- `bool` only admits `0` and `1`
- `!` admits no values at all
- References must be non-null, aligned, and point to a valid referent

Invalid Value Example

```
let raw = 2_u8;
let flag: bool = unsafe { std::mem::transmute(raw) };
println!("{flag}");
```

- The problem is not the `println!`

Invalid Value Example

```
let raw = 2_u8;
let flag: bool = unsafe { std::mem::transmute(raw) };
println!("{flag}");
```

- The problem is not the `println!`
- The problem is that the program produced an invalid `bool` at all

Never Type Example

```
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

- The never type has no values

Never Type Example

```
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

- The never type has no values
- Producing one is undefined behavior

Never Type Example

```
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

- The never type has no values
- Producing one is undefined behavior
- UB is not an exception mechanism, it is loss of semantic footing

References Carry Promises

```rust
let ptr = 0x1usize as *const u8;
let r: &u8 = unsafe { &*ptr };
println!("{}", *r);
```

- Forming a reference already claims more than “I have an address”

References Carry Promises

```
let ptr = 0x1usize as *const u8;
let r: &u8 = unsafe { &*ptr };
println!("{}", *r);
```

- Forming a reference already claims more than “I have an address”
- It claims alignment, liveness, and validity of the referent

Aliasing Example

```
fn compute(i: &u32, o: &mut u32) {
    if *i > 10 {
        *o = 1;
    }

    if *i > 5 {
        *o *= 2;
    }
}
```

```rust
fn compute(i: &u32, o: &mut u32) {
    let cached_i = *i;
    if cached_i > 10 {
        *o = 2;
    } else if cached_i > 5 {
        *o *= 2;
    }
}
```

Why This Rewrite Is Legal

- Rust references carry non-aliasing assumptions the optimizer may use

Why This Rewrite Is Legal

- Rust references carry non-aliasing assumptions the optimizer may use
- In C this sort of rewrite is much more constrained

Why This Rewrite Is Legal

- Rust references carry non-aliasing assumptions the optimizer may use
- In C this sort of rewrite is much more constrained
- In Rust unsafe code must not manufacture references that invalidate these assumptions

Packed Layout and Unaligned Access

```
#[repr(C, packed)]
struct Header {
    tag: u8,
    len: u16,
}

let header = Header { tag: 1, len: 0x1234 };
```

```rust
// let r = &header.len; // would form an unaligned reference
let len = unsafe { std::ptr::addr_of!(header.len).read_unaligned() };
println!("{len}");
```

- Packed layout often turns innocent field access into an unsafe-layout problem

```rust
// let r = &header.len; // would form an unaligned reference
let len = unsafe { std::ptr::addr_of!(header.len).read_unaligned() };
println!("{len}");
```

- Packed layout often turns innocent field access into an unsafe-layout problem
- This is one reason field-offset tricks are subtle

Raw Memory

Transmuting References

```
use std::mem;

struct BufferHolder {
    buffer: *mut i32,
    len: usize,
}
```

```rust
impl BufferHolder {
    fn new(size: usize) -> Self {
        let mut vec = vec![0; size];
        let buffer_ptr = vec.as_mut_ptr();
        mem::forget(vec);
        BufferHolder { buffer: buffer_ptr, len: size }
    }
```

```
    unsafe fn get_buffer_mut<'a>(&'a self) -> &'a mut [i32] {
        assert!(!self.buffer.is_null(), "buffer is null");
        std::mem::transmute::<(*mut i32, usize), &'a mut [i32]>((self.buffer,
self.len))
    }
}
```

Manual Drop Repair

```
impl Drop for BufferHolder {
    fn drop(&mut self) {
        unsafe {
            let _vec = Vec::from_raw_parts(self.buffer, self.len, self.len);
        }
    }
}
```

- Once we step outside ordinary ownership, destruction becomes our problem

Manual Drop Repair

```
impl Drop for BufferHolder {
    fn drop(&mut self) {
        unsafe {
            let _vec = Vec::from_raw_parts(self.buffer, self.len, self.len);
        }
    }
}
```

- Once we step outside ordinary ownership, destruction becomes our problem
- This is where double-free and leak bugs appear easily

Memory Leak Is Not UB

```rust
use std::mem;

pub fn strify(s: String) -> &'static str {
    let s_ref = s.as_str();
    let s_static_ref: &'static str = unsafe { mem::transmute(s_ref) };
    mem::forget(s);
    s_static_ref
}
```

- This leaks memory

Memory Leak Is Not UB

```rust
use std::mem;

pub fn strify(s: String) -> &'static str {
    let s_ref = s.as_str();
    let s_static_ref: &'static str = unsafe { mem::transmute(s_ref) };
    mem::forget(s);
    s_static_ref
}
```

- This leaks memory
- It does not immediately cause UB

Memory Leak Is Not UB

```rust
use std::mem;

pub fn strify(s: String) -> &'static str {
    let s_ref = s.as_str();
    let s_static_ref: &'static str = unsafe { mem::transmute(s_ref) };
    mem::forget(s);
    s_static_ref
}
```

- This leaks memory
- It does not immediately cause UB
- Safe Rust can leak too, for example through `Rc` cycles

MaybeUninit

- Uninitialized memory is not a `T` that merely holds unknown contents

MaybeUninit

- Uninitialized memory is not a `T` that merely holds unknown contents
- It is memory that has not yet become a `T`

MaybeUninit

- Uninitialized memory is not a `T` that merely holds unknown contents
- It is memory that has not yet become a `T`
- `MaybeUninit<T>` is the way to represent that honestly

MaybeUninit

- Uninitialized memory is not a `T` that merely holds unknown contents
- It is memory that has not yet become a `T`
- `MaybeUninit<T>` is the way to represent that honestly
- Without it, code tends to lie to the compiler too early

MaybeUninit Example

```rust
use std::mem::MaybeUninit;

let mut slot = MaybeUninit::<u32>::uninit();
slot.write(42);
let value = unsafe { slot.assume_init() };
println!("{value}");
```

- `assume_init` is exactly the point where you assert the invariant

MaybeUninit Example

```rust
use std::mem::MaybeUninit;

let mut slot = MaybeUninit::<u32>::uninit();
slot.write(42);
let value = unsafe { slot.assume_init() };
println!("{value}");
```

- `assume_init` is exactly the point where you assert the invariant
- Before that, the value is not a `u32` yet

Panic Safety in Unsafe Code

```
impl<T> Vec<T> {
    fn extend_map<U, F>(&mut self, us: &[U], mut f: F)
    where
        F: FnMut(&U) -> T,
    {
        self.reserve(us.len());
        let cur_len = self.len();
        unsafe { self.set_len(cur_len + us.len()); }
```

```
        let mut into = unsafe { self.as_mut_ptr().add(cur_len) };
        for u in us {
            unsafe {
                std::ptr::write(into, f(u));
                into = into.add(1);
            }
        }
    }
}
```

What Goes Wrong Here

- If `f(u)` panics halfway through, `Vec` already believes all new elements exist

What Goes Wrong Here

- If `f(u)` panics halfway through, `Vec` already believes all new elements exist
- Unwinding may then drop uninitialized elements

What Goes Wrong Here

- If `f(u)` panics halfway through, `Vec` already believes all new elements exist
- Unwinding may then drop uninitialized elements
- The bug only appears on exceptional control flow

What Goes Wrong Here

- If `f(u)` panics halfway through, `Vec` already believes all new elements exist
- Unwinding may then drop uninitialized elements
- The bug only appears on exceptional control flow
- This is why unsafe code must reason about panic paths too

Deferred Drop Is Also Tricky

- Delaying destruction can be attractive for performance

Deferred Drop Is Also Tricky

- Delaying destruction can be attractive for performance
- It can also break lifetime assumptions in subtle ways

Deferred Drop Is Also Tricky

- Delaying destruction can be attractive for performance
- It can also break lifetime assumptions in subtle ways
- If `T` contains references, “drop later” is already a semantic promise

Deferred Drop Is Also Tricky

- Delaying destruction can be attractive for performance
- It can also break lifetime assumptions in subtle ways
- If `T` contains references, “drop later” is already a semantic promise
- Safe callers can feed your unsafe structure values you did not anticipate

Safe Abstractions

Safe Wrappers Must Be Fully Safe

- A safe wrapper around unsafe internals must be correct for all safe callers

Safe Wrappers Must Be Fully Safe

- A safe wrapper around unsafe internals must be correct for all safe callers
- It is not enough that it works for intended usage

Safe Wrappers Must Be Fully Safe

- A safe wrapper around unsafe internals must be correct for all safe callers
- It is not enough that it works for intended usage
- The caller is allowed to be weird, generic, panicking, and inconvenient

Safe Wrappers Must Be Fully Safe

- A safe wrapper around unsafe internals must be correct for all safe callers
- It is not enough that it works for intended usage
- The caller is allowed to be weird, generic, panicking, and inconvenient
- If the API is safe, the burden is yours

The Whole Module Matters

- Auditing only the `unsafe {}` lines is not enough

The Whole Module Matters

- Auditing only the `unsafe {}` lines is not enough
- Safe code in the same module can violate the invariants the unsafe code relies on

The Whole Module Matters

- Auditing only the `unsafe {}` lines is not enough
- Safe code in the same module can violate the invariants the unsafe code relies on
- Constructors, mutators, `Drop`, `Clone`, and panic paths are part of the proof

The Whole Module Matters

- Auditing only the `unsafe {}` lines is not enough
- Safe code in the same module can violate the invariants the unsafe code relies on
- Constructors, mutators, `Drop`, `Clone`, and panic paths are part of the proof
- Private fields do not save you from bad safe APIs

SAFETY: Comments

```
unsafe {
    // SAFETY: `ptr` points inside `self.buf`, `index < self.len`,
    // and this slot has not been initialized yet
    std::ptr::write(ptr.add(index), value);
}
```

- The comment should name the actual invariants

SAFETY: Comments

```
unsafe {
    // SAFETY: `ptr` points inside `self.buf`, `index < self.len`,
    // and this slot has not been initialized yet
    std::ptr::write(ptr.add(index), value);
}
```

- The comment should name the actual invariants
- “SAFETY: this is safe” is useless

SAFETY: Comments

```
unsafe {
    // SAFETY: `ptr` points inside `self.buf`, `index < self.len`,
    // and this slot has not been initialized yet
    std::ptr::write(ptr.add(index), value);
}
```

- The comment should name the actual invariants
- “SAFETY: this is safe” is useless
- This is for future readers, including future you

Unsafe Traits

- `unsafe trait` means the trait has semantic obligations the compiler cannot check

Unsafe Traits

- `unsafe trait` means the trait has semantic obligations the compiler cannot check
- `Send` and `Sync` are the usual examples

Unsafe Traits

- `unsafe trait` means the trait has semantic obligations the compiler cannot check
- `Send` and `Sync` are the usual examples
- An `unsafe impl` is not a local trick, it is a global promise

Unsafe Traits

- `unsafe trait` means the trait has semantic obligations the compiler cannot check
- `Send` and `Sync` are the usual examples
- An `unsafe impl` is not a local trick, it is a global promise
- If the promise is wrong, unrelated safe code can become unsound

Keeping Unsafe Small

- Small unsafe blocks are easier to audit

Keeping Unsafe Small

- Small unsafe blocks are easier to audit
- Narrow abstraction boundaries reduce proof surface

Keeping Unsafe Small

- Small unsafe blocks are easier to audit
- Narrow abstraction boundaries reduce proof surface
- Converting raw operations into a safe local helper is usually worth it

Keeping Unsafe Small

- Small unsafe blocks are easier to audit
- Narrow abstraction boundaries reduce proof surface
- Converting raw operations into a safe local helper is usually worth it
- “Maximum cleverness” is the wrong optimization target here

Tools

Miri

- Miri interprets Rust's mid-level IR

Miri

- Miri interprets Rust's mid-level IR
- It catches many classes of UB-related mistakes during execution

Miri

- Miri interprets Rust's mid-level IR
- It catches many classes of UB-related mistakes during execution
- It is not a proof system

Miri

- Miri interprets Rust's mid-level IR
- It catches many classes of UB-related mistakes during execution
- It is not a proof system
- It is still one of the best tools we have for unsafe exploration

Running Miri

```
rustup +nightly component add miri
cargo +nightly miri test
```

- Miri should be part of the workflow for code that contains unsafe

Miri Example

```
#![feature(vec_into_raw_parts)]

struct Hehe(Vec<u8>);

impl Hehe {
    fn new() -> Self {
        let v = Vec::with_capacity(10);
        let (parts_ptr, _, parts_cap) = v.into_raw_parts();
```

```
        let v = unsafe { Vec::from_raw_parts(parts_ptr, 10, parts_cap) };
        Self(v)
    }
}

fn main() {
    let data = Hehe::new();
    println!("Uninitialized data: {}", data.0[0]);
}
```

Miri Notes

- This reads uninitialized memory

Miri Notes

- This reads uninitialized memory
- The code can look plausible in a quick review

Miri Notes

- This reads uninitialized memory
- The code can look plausible in a quick review
- Miri is very good at turning this sort of mistake into a concrete failure

Sanitizers

- Miri is not enough by itself

Sanitizers

- Miri is not enough by itself
- AddressSanitizer and ThreadSanitizer also belong in the toolbox

Sanitizers

- Miri is not enough by itself
- AddressSanitizer and ThreadSanitizer also belong in the toolbox
- Unsafe-heavy projects should run extra checks in CI

Sanitizers

- Miri is not enough by itself
- AddressSanitizer and ThreadSanitizer also belong in the toolbox
- Unsafe-heavy projects should run extra checks in CI
- Tooling does not prove soundness, but it catches real mistakes early

Further Reading

- The Rustonomicon is still the canonical long-form unsafe guide

Further Reading

- The Rustonomicon is still the canonical long-form unsafe guide
- The reference matters for layout, validity, and FFI details

Further Reading

- The Rustonomicon is still the canonical long-form unsafe guide
- The reference matters for layout, validity, and FFI details
- For nontrivial unsafe code, reading primary documentation is not optional

Closing

- Unsafe exists because some correct low-level code cannot be expressed otherwise

Closing

- Unsafe exists because some correct low-level code cannot be expressed otherwise
- The hard part is not writing the `unsafe` keyword, it is maintaining the proof around it

Closing

- Unsafe exists because some correct low-level code cannot be expressed otherwise
- The hard part is not writing the `unsafe` keyword, it is maintaining the proof around it
- The proof includes values, aliasing, lifetimes, panic paths, and module-wide invariants

Closing

- Unsafe exists because some correct low-level code cannot be expressed otherwise
- The hard part is not writing the `unsafe` keyword, it is maintaining the proof around it
- The proof includes values, aliasing, lifetimes, panic paths, and module-wide invariants
- Keep the surface small and the documentation explicit

Questions?